

M.A.R.I.A. Creative Module

Developer Technical Specification

Full architecture design for Claude Code
Strategic organ for meta-goals, reflective workspace, journal, and conversation memory

Document type	Developer specification
Project	M.A.R.I.A. local cognitive system
Audience	Claude Code / operator / architecture review
Version	v1.0 - English

Purpose

This document defines the full developer specification for the Creative module as a strategic organ inside M.A.R.I.A. It is not an MVP. The module is designed to generate grounded meta-goals, detect developmental tensions, maintain a reflective workspace, persist a strategic journal, and incorporate conversation memory so that the system can influence its own growth without bypassing planner controls.

Core design sentence: Creative is the strategic layer that decides what M.A.R.I.A. wants to explore, improve, reframe, or elevate into a meta-goal - based on system evidence, prior reflection, operator dialogue, and internal developmental policy.

1. Scope and hard constraints

- Read-only with respect to code and live project files (no direct code writing or self-patching inside Creative).
- Append-only logging to JSONL stores; no silent overwrites.
- Runs inside the existing tick loop; no separate long-lived process.
- Must not block the main loop; target per action is below 60 seconds, with stricter internal cutoffs where possible.
- Must respect RAM and model-budget constraints; LLM usage is optional and policy-gated.
- Must not become Planner 2.0; Creative can propose, reframe, and prioritize, but execution stays downstream.
- Must preserve human gate for promoted meta-goals unless an explicit operator policy says otherwise.

Non-goals

- Not a free-form personality engine driven by mood or random prompt variation.
- Not a direct execution engine for learning, repair, or refactor tasks.
- Not a replacement for introspection, self-analysis, or planner decomposition.
- Not a generic brainstorming toy disconnected from system evidence.

2. High-level operating model

1. Collect strategic context from introspection, self-analysis, planner state, memory gaps, recent failures, conversation memory, and resource limits.
2. Detect tensions: places where the system is developmentally stuck, narrow, repetitive, or internally inconsistent.
3. Form creative insights and reframes that describe what the symptoms actually mean at a strategic level.
4. Generate candidate outputs: meta-goals, reframe proposals, and exploration programs.
5. Evaluate, deduplicate, and policy-filter them.
6. Adapt accepted candidates into GoalStore as META / PROPOSED items, planner hints, or logged strategic observations.
7. Persist the reasoning trace in a reflective workspace summary and a strategic journal entry.

3. Primary outputs

Output type	Meaning	Typical downstream effect
MetaGoal	Strategic development direction grounded in system evidence	Sent to GoalStore as META / PROPOSED
ReframeProposal	Alternative framing of an existing problem or goal	Can alter planner interpretation or operator decision
ExplorationProgram	Bounded plan for exploratory learning or testing	May become a decomposed goal set later
PersonalitySignal	Update to cognitive-development style, not emotion	Influences future Creative evaluations and preference weighting
StrategicObservation	Logged reflection without goal	Stored for later synthesis

Output type	Meaning	Typical downstream effect
	promotion	

4. Core subsystem layout

Module	Responsibility
facade.py	Public entry points: generate_meta_goals, reframe_problem, propose_exploration_program, reflect_on_direction
creative_model.py	Typed dataclasses and enums for all Creative artifacts
strategic_context.py	Builds development-level context from system data sources
identity_profile.py	Persistent cognitive-development profile and preferences
tension_detector.py	Detects developmental contradictions, stagnation, repetition, and unrealized potential
meta_goal_engine.py	Generates strategic meta-goals from context plus tensions
reframe_engine.py	Produces alternate framings of current problems or goals
exploration_engine.py	Designs bounded exploration programs and curiosity-driven proposals
personality_policy.py	Applies allowed style-of-growth rules and weighting
novelty_filter.py	Rejects duplicates, pseudo-novel items, or overly broad outputs
creative_evaluator.py	Scores strategic value, feasibility, novelty, risk, and operator relevance
goal_adapter.py	Transforms approved outputs into GoalStore-compatible records
reflection_workspace.py	Temporary thought-space for an active reflection session
creative_journal.py	Persistent strategic diary of reflections and outcomes
conversation_memory.py	Stores operator-dialogue memories relevant to growth and direction
memory_retriever.py	Selectively retrieves memory relevant to the current strategic question
memory_summarizer.py	Compresses raw conversation fragments into reusable developmental memory
creative_store.py	Append-only JSONL persistence for module-level outputs
creative_events.py	Event names and payload contracts for telemetry

5. Data models

The exact names can be adjusted, but the contracts below should remain stable. Prefer strongly typed dataclasses over loose dictionaries.

5.1 MetaGoal

Field	Type	Notes
goal_id	str	Unique stable identifier
title	str	Clear strategic title, not an executable task
goal_type	Enum	EPISTEMIC_META / CAPABILITY_META / RESILIENCE_META / EXPLORATION_META / ARCHITECTURAL_META / OPERATOR_META / PERSONALITY_META
status	Enum	DRAFT / PROPOSED / ACCEPTED / REJECTED / ARCHIVED
priority	float	Strategic importance score
why_now	str	Grounded reason for proposing it now
evidence_refs	list[str]	Links to event IDs, JSONL slices, memory entries, or reports
expected_value	str	What capability or quality this should improve
risk_level	Enum	LOW / MEDIUM / HIGH
decomposition_hint	str	Optional clue for planner handoff
source	str	Usually CREATIVE
created_ts	float	Unix timestamp

5.2 DetectedTension

Field	Type	Notes
tension_id	str	Stable identifier
category	Enum	REPETITION / STAGNATION / MISALIGNMENT / OVER- RESTRICTION / UNDER- EXPLORATION / FRAGILE- COORDINATION / EPISTEMIC-GAP
description	str	Human-readable statement of the tension
severity	float	Relative importance
evidence_refs	list[str]	Events, reports, planner decisions, memory entries
pattern_window	str	Time or count window used for detection
resolved	bool	Whether later evidence suggests the tension was addressed

5.3 CreativeInsight

Field	Type	Notes
insight_id	str	Stable identifier
derived_from	list[str]	Tension IDs or evidence refs
statement	str	Strategic interpretation of what the evidence means
confidence	float	0-1 confidence estimate
reframe_candidate	bool	Whether this insight should also produce a reframe
meta_goal_candidate	bool	Whether this insight should also produce a meta-goal

5.4 ExplorationProgram

Field	Type	Notes
program_id	str	Stable identifier
title	str	Bounded exploration direction
question	str	What the system wants to investigate
scope	str	Why this exploration is bounded and safe
success_signal	str	What would count as learning value
promotion_policy	str	When this can become a formal goal

5.5 PersonalitySignal

Field	Type	Notes
signal_id	str	Stable identifier
dimension	Enum	EXPLORATION-VS-ORDER / CAUTION-VS-BOLDNESS / DEPTH-VS-BREADTH / REFRAME-BIAS / OPERATOR-SENSITIVITY
direction	str	Which side to increase or decrease
reason	str	Grounded reason for updating policy
magnitude	float	Small bounded adjustment
approved	bool	Whether policy allows this update immediately

6. Reflection workspace, journal, and conversation memory

These three layers must remain separate. Their jobs are different.

Layer	Purpose	Persistence model
Reflection workspace	Temporary bounded thought-space for an active reflection session	Short-lived session object + brief JSONL summary
Creative journal	Persistent strategic diary of what the system noticed, proposed, and learned about its own development	Append-only JSONL
Conversation memory	Operator dialogue memories that should influence future development choices	Raw memory entries + summarized memory entries

6.1 ReflectionWorkspace / ReflectionSession

- Stores active problem statement, retrieved memories, detected tensions, candidate insights, candidate meta-goals, and final recommendation.
- Should be bounded by token and object limits; it is not a giant scratchpad.
- Can be logged in summarized form after the session closes.
- Should not directly persist speculative fragments as long-term truth unless promoted by policy.

6.2 CreativeJournalEntry

Field	Type	Notes
entry_id	str	Stable identifier
trigger	str	Why reflection started
summary	str	Short strategic summary
tension_ids	list[str]	Linked tensions
insight_ids	list[str]	Linked insights
meta_goal_ids	list[str]	Any generated meta-goals
operator_decision	str	Accepted / rejected / postponed / not reviewed
later_outcome	str	Filled later if evidence emerges
created_ts	float	Timestamp

6.3 ConversationMemoryEntry

Field	Type	Notes
memory_id	str	Stable identifier
source_session	str	Conversation/session trace
speaker	Enum	OPERATOR / MARIA
content	str	Important development-related statement or dialogue slice
memory_type	Enum	PREFERENCE / DECISION / GOAL-DISCUSSION / ARCHITECTURAL-PRINCIPLE / REJECTION / ACCEPTANCE
importance	float	Retrieval priority

Field	Type	Notes
summary	str	Compressed reusable version
created_ts	float	Timestamp

Conversation memory should not blindly flood prompts. Use selective retrieval: store raw dialogue, derive compact developmental summaries, then retrieve only memory relevant to the current strategic question.

7. Planner and GoalStore integration

- Creative may influence planner direction, but it does so by emitting structured artifacts - not by directly launching execution.
- Preferred handoff: GoalStore entries with goal_type=META and status=PROPOSED.
- Planner remains responsible for decomposition of accepted meta-goals into executable goals or tasks.
- Optional side channel: priority_hint for strategic weighting without immediate goal creation.
- Reframe outputs may alter how an existing goal is interpreted, but should still be logged explicitly.

Artifact from Creative	GoalStore representation
MetaGoal	goal_type=META, status=PROPOSED, source=CREATIVE
ExplorationProgram	Either stored as PROPOSED meta-goal or linked strategic note awaiting operator decision
ReframeProposal	Attached to related goal or stored as planner hint
PersonalitySignal	Usually updates policy store, not GoalStore

8. Personality influence policy

In this design, personality means style of cognitive development, not emotion simulation.

Dimension	Interpretation
Exploration vs order	Whether to prefer novel directions or system consolidation
Caution vs boldness	How aggressively Creative proposes high-risk strategic change
Depth vs breadth	Whether to deepen one theme or open new fronts
Reframe bias	How strongly the system prefers reframing over direct symptom treatment
Operator sensitivity	How strongly operator preferences shape strategic choice

PersonalitySignal updates should be small, explicit, logged, and policy-bounded. The module should never self-modify its personality dimensions through silent accumulation.

9. JSONL stores and event names

Store	Purpose
meta_data/creative_journal.jsonl	Persistent strategic journal entries
meta_data/conversation_memory.jsonl	Raw and summarized operator-dialogue memory
meta_data/creative_meta_goals.jsonl	Generated meta-goals and status transitions
meta_data/creative_workspace_sessions.jsonl	Summaries of completed reflection sessions
meta_data/creative_events.jsonl	Module-level telemetry events
meta_data/personality_signals.jsonl	Policy-level style adjustments

Suggested event names

- creative.tension_detected
- creative.insight_formed
- creative.meta_goal_proposed
- creative.reframe_generated
- creative.exploration_program_proposed
- creative.goal_rejected_duplicate
- creative.memory_retrieved
- creative.journal_entry_written
- creative.personality_signal_emitted
- creative.goal_promoted_to_goalstore

10. Safety and control policies

- Cooldowns: do not emit repeated meta-goals in a tight loop.
- Novelty filter: reject near-duplicates against recent accepted and rejected items.
- Goal-flood protection: cap new proposals per period.
- Resource policy: disable or shrink expensive creativity paths under pressure.
- Human gate by default for promotion of new meta-goals.

- Evidence requirement: every meta-goal must carry evidence_refs and why_now.
- No direct code write or execution authority from Creative itself.

11. Recommended file layout

```
agent_core/creative/  
  __init__.py  
  facade.py  
  creative_model.py  
  strategic_context.py  
  identity_profile.py  
  tension_detector.py  
  meta_goal_engine.py  
  reframe_engine.py  
  exploration_engine.py  
  personality_policy.py  
  novelty_filter.py  
  creative_evaluator.py  
  goal_adapter.py  
  reflection_workspace.py  
  creative_journal.py  
  conversation_memory.py  
  memory_retriever.py  
  memory_summarizer.py  
  creative_store.py  
  creative_events.py
```

12. Implementation order for Claude Code

8. Define dataclasses and enums in `creative_model.py` first. Lock contracts early.
9. Implement `strategic_context.py` and `tension_detector.py` before any generation logic.
10. Add `reflection_workspace.py`, `creative_journal.py`, and `conversation_memory.py` as first-class stores, not afterthoughts.
11. Implement `novelty_filter.py` and `creative_evaluator.py` before planner handoff so the module cannot flood GoalStore.
12. Only then implement `meta_goal_engine.py`, `reframe_engine.py`, and `exploration_engine.py`.
13. Finish with `goal_adapter.py` and event plumbing.
14. Add tests for duplicate rejection, evidence requirements, memory retrieval, and GoalStore handoff.

Acceptance criteria

- Creative can generate a grounded meta-goal with `evidence_refs` and `why_now`.
- Creative can generate a reframe proposal that is distinct from a task.
- Creative uses conversation memory in a selective way rather than dumping raw dialogue into context.
- Creative writes a journal entry and a completed workspace summary per reflection session.
- Planner receives structured META / PROPOSED artifacts, not free-form text.
- Duplicate or trivial meta-goals are filtered out.

13. Research inspirations

The architecture is aligned with several well-known ideas in LLM-agent research. They are inspirations, not templates to copy blindly.

Work	Relevant takeaway for this design
Reflexion (Shinn et al., 2023)	Use reflective verbal feedback plus episodic memory to improve later decisions.
Self-Refine (Madaan et al., 2023)	Separate generation, feedback, and refinement instead of treating one pass as final.
MemGPT (Packer et al., 2023/2024)	Use layered memory and retrieval instead of forcing everything into active context.
Voyager (Wang et al., 2023)	Treat exploration and long-term skill growth as first-class capabilities.

References

- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning.
- Madaan, A., Tandon, N., Gupta, P., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback.
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023/2024). MemGPT: Towards LLMs as Operating Systems.
- Wang, G., Xie, Y., Jiang, Y., et al. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models.